

Prompt 1:

“ Correct this prompt and add any missing elements that needs to be mentioned according to the lessons given earlier and the consigne

Prompt à corriger:

Je dois préparer le cahier des charges de notre projet d’interopérabilité en santé.

Nous avons choisi le **sujet A : Admission d’un patient**.

L’objectif est de réaliser un petit prototype capable de récupérer de vraies données depuis un serveur FHIR, de les afficher de manière compréhensible pour un utilisateur hospitalier, puis de transformer les informations utiles vers un message **HL7 v2** correspondant à une admission.

Pour les tests, nous utiliserons le serveur HAPI FHIR :

<https://hapi.fhir.org/baseR4>

Swagger :

<https://hapi.fhir.org/baseR4/swagger-ui/>

Nous travaillons avec **FHIR R4**.

Pour le moment, je souhaite uniquement construire le **cahier des charges fonctionnel et technique**. Il ne faut pas encore développer l’application.

Le projet doit rester simple et réalisable dans un **MVP d’environ 2 heures de développement**.

1. Contexte et besoin métier

Commence par expliquer simplement le besoin métier.

Dans notre cas, un utilisateur hospitalier doit pouvoir retrouver un patient présent sur le serveur FHIR et consulter les informations utiles concernant son admission.

Il faut préciser :

- qui est l’utilisateur principal ;
- ce qu’il cherche à faire ;
- le résultat attendu ;
- le scénario normal ;
- les principaux cas d’erreur ;
- les critères permettant de dire que le prototype fonctionne correctement.

Il faut bien distinguer le **besoin métier** de la **solution technique** utilisée pour y répondre.

2. Ressources FHIR nécessaires

Identifier les ressources FHIR réellement nécessaires pour représenter le scénario d'admission.

Je pense notamment à :

- Patient
- Encounter

mais il faut vérifier que ces deux ressources suffisent réellement.

Pour chaque ressource retenue, préciser :

- son rôle dans le scénario ;
- les champs réellement utiles ;
- les liens entre les ressources ;
- les données nécessaires à l'affichage ;
- les données nécessaires pour construire le message HL7 v2 ;
- les champs facultatifs dont l'absence doit être gérée.

Il faut également distinguer ce qui appartient au **standard FHIR R4**, ce qui pourrait venir d'un profil ou d'un Implementation Guide, et ce qui correspond simplement à un choix fait pour notre prototype.

Ne pas annoncer une conformité à FR Core, US Core ou à un autre profil si elle n'a pas été vérifiée.

3. Communication avec le serveur FHIR

Définir les requêtes nécessaires pour réaliser le scénario.

Par exemple :

GET /Patient/{id}

et éventuellement :

GET /Encounter?patient={id}

Pour chaque interaction, préciser :

- la ressource concernée ;
- l'objectif de la requête ;
- les paramètres utilisés ;
- le type de réponse attendu ;
- le traitement fait par l'application ;
- les erreurs possibles.

Comme les recherches FHIR dépendent des capacités du serveur, ne pas supposer qu'un SearchParameter fonctionne automatiquement.

Les éléments importants devront être vérifiés dans le **CapabilityStatement**, Swagger ou avec Postman.

Le serveur HAPI étant public et partagé, le prototype ne doit pas dépendre d'un identifiant de patient supposé rester disponible en permanence.

4. Transformation vers HL7 v2

Le format de sortie demandé est **HL7 v2**.

Pour une admission, je pense utiliser un message **ADT^A01**, mais il faut confirmer que ce choix est bien adapté au scénario et expliquer pourquoi.

Pour le MVP, garder uniquement les segments réellement utiles, probablement :

- MSH
- PID
- PV1

Pour les principales données utilisées, construire un tableau de correspondance de la forme :

Donnée FHIR → champ HL7 v2 → règle de transformation → obligatoire ou facultatif → comportement si absent → perte éventuelle d'information

Il ne faut jamais inventer une information pour remplir un champ HL7 si cette information n'existe pas dans les ressources FHIR récupérées.

5. Mapping sémantique

Le projet doit aussi montrer qu'on ne fait pas uniquement une transformation de structure, mais qu'on comprend l'aspect **sémantique**.

Je souhaite donc intégrer au moins un exemple simple de mapping terminologique.

Un exemple possible serait :

`Patient.gender`

avec les valeurs FHIR :

`male / female / other / unknown`

vers la valeur correspondante dans `PID-8` en HL7 v2.

Il faut vérifier les valeurs autorisées des deux côtés et montrer clairement :

code ou valeur source → règle de transformation → code ou valeur cible

Il faut aussi expliquer ce qu'on fait si aucune correspondance exacte n'existe, afin d'éviter de transformer silencieusement une donnée de manière incorrecte.

6. ReEIF appliqué au projet

Faire une petite analyse du projet selon les différentes couches du ReEIF :

- juridique / réglementaire ;
- politique / gouvernance ;
- processus de soin ;
- information ;
- applications ;
- infrastructure SI.

Je ne veux pas une définition théorique de chaque couche.

Il faut simplement expliquer, pour notre cas d'admission d'un patient, quelle question ou quel risque se pose à chaque niveau.

7. Architecture du prototype

L'architecture doit rester volontairement simple.

Par exemple :

Utilisateur → Interface → Service FHIR → Serveur HAPI FHIR

puis :

Ressources FHIR → logique métier → transformation HL7 v2

L'application finale devra idéalement permettre de voir :

1. les informations métier de l'admission ;
2. les ressources FHIR récupérées ;
3. le message HL7 v2 généré ;
4. le mapping terminologique utilisé.

Pas besoin de base de données locale, de microservices ou d'authentification complexe pour ce prototype.

8. Contraintes importantes

Le cahier des charges doit respecter les contraintes suivantes :

- aucune donnée Patient ou Encounter ne doit être codée en dur ;
- les informations affichées doivent réellement venir du serveur FHIR ;
- l'application doit fonctionner avec plusieurs patients compatibles, pas seulement un exemple précis ;
- une donnée absente ne doit jamais être inventée ;

- les erreurs principales doivent être gérées ;
- il doit être possible de retrouver l'origine FHIR d'une information transformée en HL7 v2 ;
- l'accès FHIR, la logique métier, la transformation HL7 et l'interface doivent rester séparés ;
- le projet doit rester réalisable en environ 2 heures.

9. Vérifications avant de développer

Avant de commencer le développement, préparer une checklist pour vérifier :

- la version FHIR du serveur ;
- l'accès au `CapabilityStatement` ;
- les ressources disponibles ;
- les profils éventuellement déclarés ;
- les SearchParameters réellement supportés ;
- la présence d'Encounter associés à certains patients ;
- la manière dont `Encounter.subject` référence le patient ;
- la présence des informations nécessaires à la conversion HL7 v2 ;
- le comportement lorsqu'une recherche ne retourne aucun résultat ;
- la présence ou non d'une authentification ;
- les principales erreurs HTTP possibles.

Lorsqu'une information n'est pas encore confirmée, l'indiquer clairement comme :

À VÉRIFIER

10. Plan de tests

Préparer ensuite un plan de tests simple, d'abord avec Swagger ou Postman, puis dans l'application.

Tester au minimum :

- recherche d'un patient existant ;
- patient inexistant ;
- récupération d'un Patient ;
- recherche des Encounter associés ;
- patient sans Encounter ;
- absence d'un champ facultatif ;
- traitement correct d'un `Bundle` ;
- lecture de `Bundle.entry[] .resource` ;
- vérification de la relation entre `Encounter.subject` et le patient ;
- génération du message HL7 v2 ;
- mapping terminologique ;
- erreurs du serveur ;
- test avec au moins un deuxième patient afin de vérifier que l'application n'est pas construite uniquement pour un cas particulier.

Présenter les tests sous la forme :

Précondition | Action | Résultat attendu

11. Démarche de vérification et d'itération

Je veux aussi que le cahier des charges montre que le projet va être construit progressivement et que nos premières hypothèses peuvent être corrigées.

Pour les points importants, suivre la logique :

Hypothèse → Vérification → Résultat → Écart → Correction → Décision

Je vois par exemple les étapes suivantes :

Itération 1 : compréhension du serveur

Tester HAPI et vérifier FHIR R4, les ressources disponibles et les paramètres de recherche.

Itération 2 : modèle métier et FHIR

Vérifier si Patient et Encounter suffisent réellement et si les champs attendus sont présents.

Itération 3 : transformation HL7 v2

Tester le mapping avec de vraies données récupérées sur le serveur.

Itération 4 : prototype

Connecter l'interface aux données FHIR.

Itération 5 : validation

Tester plusieurs patients et corriger les hypothèses qui se révèlent fausses.

Prévoir aussi quelques exemples de corrections possibles, par exemple :

- un SearchParameter que nous pensions disponible ne fonctionne pas ;
- un champ attendu dans Encounter n'est pas renseigné ;
- le mapping entre deux codes n'est pas exact ;
- une donnée facultative manque ;
- la structure du Bundle reçu est différente de ce que nous avons prévu.

12. Format attendu du cahier des charges

Le cahier des charges final doit rester clair, concis et directement utilisable pour développer le prototype.

Je souhaite l'organiser ainsi :

1. Contexte et objectif
2. Besoin métier
3. Utilisateur et scénario
4. Périmètre fonctionnel et hors périmètre
5. Critères d'acceptation

6. Spécifications FHIR
7. Interactions avec le serveur HAPI
8. Spécification HL7 v2 cible
9. Mapping FHIR → HL7 v2
10. Mapping sémantique
11. Analyse ReEIF
12. Architecture
13. Gestion des erreurs et traçabilité
14. Plan de tests
15. Hypothèses à vérifier
16. Limites et pertes d'information
17. Itérations et corrections

Avant de finaliser le cahier des charges, vérifie également :

- quelles hypothèses ne sont pas encore confirmées ;
- s'il existe des incohérences ;
- si certaines fonctionnalités sont trop ambitieuses pour un développement de 2 heures ;
- si les éléments essentiels du sujet sont bien couverts ;
- quelles simplifications seraient utiles pour garder un MVP réaliste.

À la fin, ajouter une section :

Points à vérifier avant de figer le cahier des charges

avec une petite checklist permettant de décider :

GO / NO-GO pour commencer le développement.”

Réponse:

Je prépare le **cahier des charges d'une évaluation d'interopérabilité en santé**.

Le sujet choisi est :

A — Admission d'un patient

L'objectif est de concevoir puis développer un prototype réellement connecté à un serveur FHIR, sans données métier codées en dur, puis de transformer les données utilisées vers un **format cible HL7 v2**.

Le serveur de test utilisé est :

<https://hapi.fhir.org/baseR4>

Swagger :

<https://hapi.fhir.org/baseR4/swagger-ui/>

Notre cible est **FHIR R4**.

Avant de figer toute hypothèse sur la version, les ressources disponibles ou les paramètres de recherche, vérifie ce qui doit être confirmé via le [CapabilityStatement](#), Swagger ou des tests Postman.

À ce stade, je veux uniquement construire le **cahier des charges fonctionnel et technique**.

Ne génère pas encore le code de l'application.

Le cahier des charges doit démontrer une compréhension réelle de l'interopérabilité : une connexion technique ne suffit pas. Il faut distinguer **objectif métier, infrastructure, interaction, structure, sémantique, workflow, gouvernance, transformation et validation**.

L'application finale devra permettre à un utilisateur hospitalier de rechercher ou sélectionner un patient présent sur le serveur FHIR, de récupérer les informations pertinentes liées à son admission, puis :

- afficher une vue métier lisible de l'admission ;
 - afficher les ressources FHIR réellement utilisées ;
 - produire une conversion vers un message HL7 v2 adapté à une admission ;
 - démontrer au moins un mapping ou alignement terminologique source → cible ;
 - gérer les erreurs principales ;
 - rester réalisable dans un MVP de **2 heures de développement**.
-

1. Commencer par le besoin métier

Définis clairement :

- le besoin métier lié à l'admission d'un patient ;
- l'utilisateur principal ;
- le résultat attendu pour cet utilisateur ;
- la différence entre l'objectif métier et la solution technique ;
- un scénario nominal simple ;
- les principaux scénarios d'erreur ;
- des critères d'acceptation mesurables.

Le scénario doit rester réaliste pour un prototype de 2 heures.

Ne propose pas de fonctionnalité non indispensable au MVP.

2. Définir précisément le périmètre FHIR

Identifie les ressources FHIR strictement nécessaires pour représenter une admission.

Je pense a priori à :

- Patient
- Encounter

mais ne considère pas cette hypothèse comme automatiquement correcte.

Pour chaque ressource retenue, indique :

- son rôle métier ;
- les éléments réellement utiles au cas d'usage ;
- les références entre ressources ;
- les champs nécessaires à l'affichage métier ;
- les champs nécessaires à la conversion HL7 v2 ;
- les champs optionnels dont l'absence doit être gérée.

Distingue explicitement :

- ce qui vient du **standard FHIR R4** ;
- ce qui viendrait éventuellement d'un **profil ou Implementation Guide** ;
- ce qui constitue une **décision propre à notre prototype**.

Ne prétends pas à une conformité FR Core, US Core ou autre profil tant qu'elle n'est pas démontrée.

Si un profil est proposé, explique pourquoi il est pertinent.

3. Définir les interactions avec le serveur FHIR

Propose uniquement les interactions nécessaires au scénario :

- lecture ;
- recherche ;
- création ou mise à jour uniquement si elles sont réellement nécessaires.

Pour chaque interaction, précise :

- ressource concernée ;
- objectif métier ;
- requête REST envisagée ;
- paramètres de recherche ;
- type de réponse attendu (Patient, Encounter, Bundle, etc.) ;
- traitement attendu côté application ;
- principaux cas d'erreur.

Examine par exemple :

GET /Patient/{id}

GET /Encounter?patient={id}

mais ne suppose pas qu'un paramètre de recherche est supporté.

Indique ce qui devra être vérifié via le [CapabilityStatement](#), Swagger ou Postman.

Comme HAPI est un serveur public partagé, ne base pas le scénario sur un identifiant de ressource supposé permanent.

4. Définir le format cible HL7 v2

Le format cible demandé est **HL7 v2**.

Détermine le type de message le plus pertinent pour une admission.

[ADT^A01](#) semble être un candidat logique, mais je veux que tu le **confirmes et le justifies** avant de le retenir.

Définis un MVP du message avec uniquement les segments nécessaires.

Vérifie notamment la pertinence de :

- [MSH](#)
- [PID](#)
- [PV1](#)

Pour chaque donnée transformée, construis une table :

[élément FHIR source](#) → [champ HL7 v2 cible](#) → [règle de transformation](#) → [obligatoire/facultatif](#) → [comportement si absent](#) → [perte éventuelle d'information](#)

Ne complète jamais artificiellement un champ HL7 v2 avec une donnée métier inexistante dans FHIR.

5. Identifier un véritable mapping sémantique

L'évaluation exige une démonstration d'alignement ou de changement de terminologie.

Propose au moins un mapping simple, visible et défendable.

Exemple candidat :

Patient.gender

valeurs FHIR :

male / female / other / unknown

vers la représentation adaptée dans PID-8.

Avant de retenir ce mapping :

- vérifie le système de codes FHIR source ;
- vérifie les valeurs HL7 v2 cibles ;
- vérifie qu'il s'agit bien d'un mapping sémantique et non d'un simple changement d'affichage.

Présente le résultat sous la forme :

système/code source → règle de transformation → système/code cible

Explique :

- pourquoi ce mapping est sémantique ;
 - ce qui se passe si aucune correspondance exacte n'existe ;
 - comment éviter une traduction silencieuse incorrecte ;
 - toute perte ou approximation éventuelle.
-

6. Appliquer concrètement le ReEIF

Analyse brièvement le prototype selon les six couches :

- juridique / réglementaire ;
- politique / gouvernance ;
- processus de soin ;
- information ;
- applications ;
- infrastructure SI.

Pour chaque couche :

- applique-la uniquement au cas d'usage « admission d'un patient » ;
 - indique le risque ou la question principale ;
 - évite les définitions théoriques génériques.
-

7. Définir une architecture minimale

Propose une architecture volontairement simple, par exemple :

Utilisateur → Interface → Service FHIR → Serveur HAPI FHIR

puis :

Ressources FHIR → couche métier → transformateur HL7 v2

L'application finale devra contenir au minimum quatre vues ou zones :

1. vue métier de l'admission ;
2. vue des ressources FHIR réellement reçues ;
3. vue du message HL7 v2 généré ;
4. vue du mapping terminologique source → cible.

Ne propose pas :

- base de données locale ;
- authentification complexe ;
- microservices ;
- architecture inutilement sophistiquée,

sauf justification indispensable.

8. Intégrer les contraintes du sujet

Le CdC doit intégrer explicitement :

- aucune donnée métier **Patient** ou **Encounter** codée en dur ;
 - les données affichées proviennent réellement du serveur FHIR ;
 - le prototype doit être générique et fonctionner avec plusieurs ressources compatibles ;
 - aucune donnée absente ne doit être inventée ;
 - gestion minimale des erreurs ;
 - traçabilité entre donnée FHIR source et donnée HL7 cible ;
 - séparation entre accès FHIR, logique métier, transformation et interface ;
 - périmètre réalisable en 2 heures.
-

9. Préparer les vérifications avant développement

Construis une checklist de vérification comprenant notamment :

- version FHIR réellement exposée ;
- **CapabilityStatement** accessible ;

- présence des ressources nécessaires ;
- profils déclarés ;
- paramètres de recherche réellement supportés ;
- présence d'**Encounter** associés à certains patients ;
- structure réelle des références **Patient** ↔ **Encounter** ;
- disponibilité des données nécessaires au HL7 v2 ;
- comportement du serveur si aucune ressource n'est trouvée ;
- éventuelle authentification ;
- comportement en cas d'erreur HTTP.

Toute hypothèse non vérifiée doit être marquée :

À VÉRIFIER

et ne doit jamais être présentée comme un fait.

10. Définir le plan de tests

Prépare un plan de tests minimal à exécuter d'abord dans Swagger ou Postman, puis dans l'application.

Inclure au minimum :

- recherche d'un patient existant ;
- patient inexistant ;
- lecture d'un **Patient** ;
- recherche des **Encounter** associés ;
- patient sans **Encounter** ;
- champ optionnel absent ;
- réponse **Bundle** correctement interprétée ;
- lecture de **Bundle.entry[].resource** ;
- relation correcte entre **Encounter.subject** et le patient ;
- génération correcte du message HL7 v2 ;
- mapping terminologique correct ;
- gestion des erreurs ;
- test avec un deuxième patient pour vérifier que l'application n'est pas codée pour un seul cas.

Présente chaque test sous la forme :

Précondition | **Action** | **Résultat attendu**

11. Prévoir une démarche d'itération et de correction

Le cahier des charges doit montrer une démarche explicite de vérification.

Pour chaque élément important, utilise la logique :

Hypothèse → Vérification → Résultat → Écart éventuel → Correction →
Décision finale

Prévois notamment les itérations suivantes :

Itération 1 — Compréhension du serveur

Tester le serveur HAPI et vérifier les hypothèses sur FHIR R4, les ressources et les SearchParameters.

Itération 2 — Modèle métier/FHIR

Vérifier que **Patient** et **Encounter** suffisent et que les champs attendus sont réellement présents.

Itération 3 — Transformation HL7 v2

Tester le mapping sur une vraie ressource du serveur.

Itération 4 — Prototype

Connecter l'interface aux données réelles.

Itération 5 — Validation

Tester avec plusieurs patients et corriger les hypothèses incorrectes.

Donne au moins trois exemples de corrections possibles, par exemple :

- SearchParameter non supporté ;
- champ Encounter attendu absent ;
- mapping non exact ;
- donnée optionnelle manquante ;
- requête retournant un **Bundle** différent de l'hypothèse.

12. Règles de pilotage de l'IA

Tu peux proposer, mais tu ne dois pas décider silencieusement à ma place.

Lorsque plusieurs solutions sont possibles :

1. présente brièvement les alternatives ;
2. recommande la plus simple pour le MVP ;
3. justifie ce choix.

Avant d'affirmer une information normative concernant FHIR ou HL7 v2 :

- indique ce qui doit être vérifié ;
- distingue ce qui est certain de ce qui est supposé ;
- ne mélange pas FHIR international, profil national et format cible HL7 v2 ;
- n'invente pas un champ ou un mapping parce qu'il « semble logique ».

Si une information est incertaine, marque-la explicitement :

À VÉRIFIER

Livrable attendu

Produis un cahier des charges concis, structuré et directement exploitable par une équipe de développement.

Structure-le avec les sections suivantes :

1. Contexte et objectif
 2. Besoin métier
 3. Utilisateur et scénario
 4. Périmètre fonctionnel / hors périmètre
 5. Critères d'acceptation
 6. Spécifications FHIR
 7. Interactions avec le serveur HAPI
 8. Spécification HL7 v2 cible
 9. Table de mapping FHIR → HL7 v2
 10. Mapping sémantique
 11. Analyse ReEIF
 12. Architecture
 13. Gestion des erreurs et traçabilité
 14. Plan de tests
 15. Hypothèses à vérifier
 16. Limites et pertes d'information
 17. Plan d'itérations et corrections
-

Revue critique avant version finale

Avant de produire la version finale du cahier des charges :

1. identifie les hypothèses non vérifiées ;
2. détecte les incohérences éventuelles ;
3. signale les éléments trop complexes pour 2 heures ;
4. vérifie que toutes les exigences de l'évaluation sont couvertes ;

5. propose les simplifications nécessaires.

Ensuite seulement, produis la **version corrigée et finale du cahier des charges**.

Termine par :

Points à vérifier avant de figer le CdC

puis une checklist :

GO / NO-GO pour démarrer le développement.

Prompt 2:

Génère moi un cahier des charges selon la prompt corrigée en doc

Sur github:

Cahier_des_charges_Admission_Patient_FHIR_HL7v2.docx